

20.1 BFS & DFS

In [Chapter 19.4](#), we developed DFS (Depth First Search) Traversal for graphs. In DFS, we visit down the entire lineage of our first child before we even begin to look at our second child - we literally search *depth first*.

Here, we will talk about BFS (Breadth First Search) (also known as Level Order Traversal). In BFS, we visit all of our immediate children before continuing on to any of our grandchildren. In other words, we visit all nodes 1 edges from our source. Then, all nodes 2 edges from our source, etc.

The pseudocode for BFS is as follows:

```
Initialize the fringe, an empty queue
  add the starting vertex to the fringe
  mark the starting vertex
  while fringe is not empty:
    remove vertex v from the fringe
    for each neighbor n of vertex v:
      if n is not marked:
        add n to fringe
        mark n
        set edgeTo[n] = v
        set distTo[n] = distTo[v] + 1
```

A *fringe* is just a term we use for the data structure we are using to store the nodes on the frontier of our traversal's discovery process (the next nodes it is waiting to look at). For BFS, we use a queue for our fringe.

`edgeTo[...]` is a map that helps us track how we got to node `n`; we got to it by following the edge from `v` to `n`.

`distTo[...]` is a map that helps us track how far `n` is from the starting vertex. Assuming that each edge is worth a distance of 1, then the distance to `n` is just one more than the distance to get to `v`. Why? We can use the way we know how to get to `v`, then pay one more to arrive at `n` via the edge that necessarily exists between `v` and `n` (it must exist since in the for loop header, `n` is defined as a neighbor of `v`).

This [slide deck](#) illustrates how this pseudocode can be carried out on an example graph.

DFS vs BFS

► **Question 18.1:** What graph traversal algorithm uses a stack rather than a queue for its fringe?

Note however that DFS and BFS differ in more than just their fringe data structure. They differ in the order of marking nodes. For DFS we mark nodes only once we visit a node - aka pop it from the fringe. As a result, it's possible to have multiple instances of the same node on the stack at a time if that node has been queued but not visited yet. With BFS we mark nodes as soon as we add them to the fringe so this is not possible.

Recursive DFS implements this naturally via the recursive stack frames; iterative DFS implements it manually:

```
Initialize the fringe, an empty stack
push the starting vertex on the fringe
while fringe is not empty:
    pop a vertex off the fringe
    if vertex is not marked:
        mark the vertex
        visit vertex
        for each neighbor of vertex:
            if neighbor not marked:
                push neighbor to fringe
```

20.2 Representing Graphs

We will discuss our choice of **API**, and also the **underlying data structures** used to represent the graph. Our decisions can have profound implications on our *runtime*, *memory usage*, and *difficulty of implementing various graph algorithms*.

Graph API

An API (Application Programming Interface) is a list of methods available to a user of our class, including the method signatures (what arguments/parameters each function accepts) and information regarding their behaviors. You have already seen APIs from the Java developers for the classes they provide, such as the [Deque](#).

For our Graph API, let's use the common convention of assigning each unique node to an integer number. This can be done by maintaining a map which can tell us the integer assigned to each original node label. Doing so allows us to define our API to work with integers specifically, rather than introducing the need for generic types.

We can then define our API to look something like this perhaps:

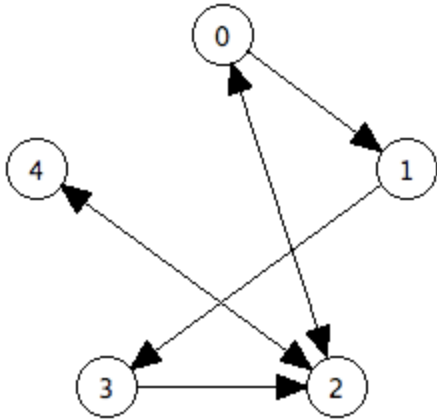
```
public class Graph {
    public Graph(int V):           // Create empty graph with v vertices
    public void addEdge(int v, int w): // add an edge v-w
    Iterable<Integer> adj(int v):    // vertices adjacent to v
    int V():                       // number of vertices
    int E():                       // number of edges
    ...
}
```

Clients (people who wish to use our Graph data structure), can then use any of the functions we provide to implement their own algorithms. The methods we provide can have a significant impact on how easy/difficult it may be for our clients to implement particular algorithms.

Now that we know how to draw a graph on paper and understand the basic concepts and definitions, we can now consider how a graph should be represented inside of a computer. We want to be able to get quick answers for the following questions about a graph:

- Are given vertices u and v adjacent?
- Is vertex v incident to a particular edge e ?
- What vertices are adjacent to v ?
- What edges are incident to v ?

Imagine that we want to represent a graph that looks like this:

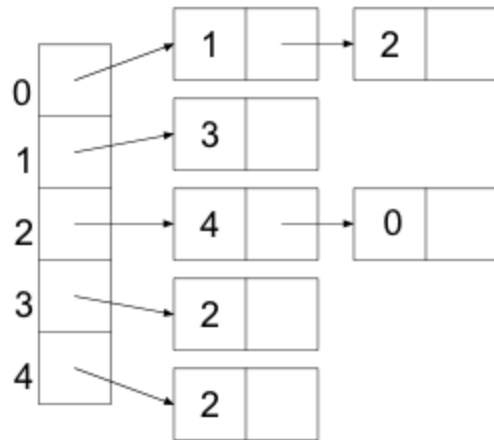


One data structure we could use to implement this graph is called an *array of adjacency lists*.

The Adjacency List

In an adjacency list, an array is created that has the same size as the number of vertices in the graph. Each position in the array represents one of the vertices in the graph. Each of these positions point to a list. These lists are called adjacency lists, as each element in the list represents a neighbor of the vertex.

The array of adjacency lists that represents the above graph looks like this:



Another data structure we could use to represent the edges in a graph is called an *adjacency matrix*.

The Adjacency Matrix

In this data structure, we have a two dimensional array of size $N \times N$ (where N is the number of vertices) which contains boolean values. The (i, j) th entry of this matrix is true when there is an edge from i to j and false when no edge exists. Thus, each vertex has a row and a column in the matrix, and the value in that table says true or false whether or not that edge exists.

The adjacency matrix that represents the above graph looks like this:

	0	1	2	3	4
0	F	T	T	F	F
1	F	F	F	T	F
2	T	F	F	F	T
3	F	F	T	F	F
4	F	F	T	F	F

Efficiency

Your choice of underlying data structure can impact the runtime and memory usage of your graph. This table from the [slides](#) summarizes the efficiencies of each representation for various operations. It is strongly not recommended to directly just copy this on to your cheatsheet for the exams without taking the time to first understand where and how these bounds fundamentally came to be. The lecture contains walkthroughs explaining the rationale in detail behind several of these cells.

Further, DFS/BFS on a graph backed by adjacency lists runs in $O(V+E)$, while on a graph backed by an adjacency matrix runs in $O(V^2)$. See the [slides](#) for help in understanding why.

20.3 Summary

Graph Traversals Overview

- The same traversals that we used on trees can be generalized to graphs. Given a source vertex, we can visit vertices in:
 - DFS preorder: the order in which DFS is called on each vertex.
 - DFS postorder: the order in which DFS returns from each vertex.
 - BFS: the order of distance from the source node (this is level-order in trees).

BFS

- Unlike DFS, BFS has a natural solution that is iterative, not recursive. BFS visits a source vertex s , then every vertex at distance 1 from s , then every vertex at distance 2 from s , and so on.
- BFS uses a *fringe* of vertices that are next to be explored. In BFS, this fringe is a queue. We enqueue new vertices at the end, and dequeue vertices to visit from the front.
- BFS can be used to solve the shortest paths problem, given that we want to minimize the number of edges from source to each other vertex. If we want to recover the shortest path from BFS, we need to track the `edgeTo` each vertex during our traversal.

Graph Implementation

- The choice of API for a graph determines how clients must write their code. Certain APIs make some tasks easier and other tasks harder. The choice of API can also affect runtime and memory.
- Choice of graph implementations include adjacency matrices, lists of edges, and adjacency lists. An adjacency matrix is a 2D boolean array indicating whether any pair of vertices are adjacent. A list of edges is a collection of all edges in the graph.
- The most common approach to graph representation is an adjacency list. In this representation, we maintain a array of lists indexed by vertex number; each index stores all vertices connected to the given vertex.

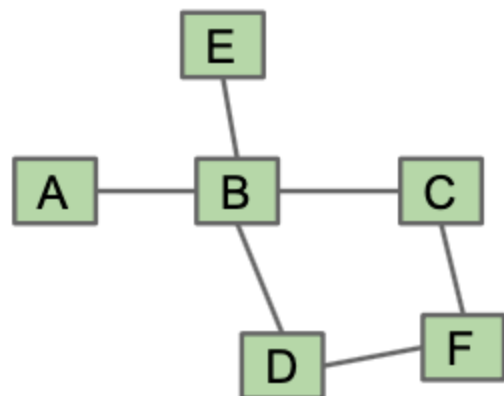
20.4 Exercises

Factual

1. Suppose we want to find the vertex with minimal degree (fewest neighbors). Is an adjacency matrix or adjacency list better?

► Problem 1

Procedural



1. For the graph above, write the representation for the adjacency matrix representation.
2. Repeat (1) for the list of edges representation.
3. Repeat (1) for the adjacency list representation.
4. Run BFS starting from A for the graph above, and list the order in which vertices are visited. Break ties alphabetically.
5. Run DFS starting from A for the graph above, and list the postorder/postorder traversals. Break ties alphabetically.

- Problem 1
- Problem 2
- Problem 3
- Problem 4
- Problem 5

Metacognitive

1. Suppose we find some shortest path from a to b using BFS. Consider a vertex c that is on the path between a and b. What can we say about the shortest path from c to b?
2. Problem 4c from the [Spring 2015 Midterm 2](#).

- Problem 1
- Problem 2

20. Graph Traversals and Implementations